LeetCode 242: Valid Anagram

💡 Optimal Logic (Frequency Count)

Since the strings contain only lowercase English letters (`a-z`), we can use an array of size **26** to store character frequencies.

Steps

1. If the lengths of both strings are different, return **false**.
2. Create an integer array `count[26]` initialized to `0`.
3. Traverse both strings simultaneously:
    - Increment the count for the character in `s`.
    - Decrement the count for the character in `t`.
4. After the traversal, if every element in the frequency array is `0`, the strings are anagrams.
5. Otherwise, return `false`.

⏱ Time Complexity

O(n)

💾 Space Complexity

O(1) (Fixed-size array of 26)

🔹 C

```
#include <stdbool.h>
#include <string.h>
bool isAnagram(char* s, char* t) {
    if (strlen(s) != strlen(t))
        return false;

    int count[26] = {0};
    for (int i = 0; s[i] != '\0'; i++) {
        count[s[i] - 'a']++;
        count[t[i] - 'a']--;
    }

    for (int i = 0; i < 26; i++) {
        if (count[i] != 0)
            return false;
    }

    return true;
```

```
}
```

□ C++

```cpp
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length() != t.length())
            return false;

        int count[26] = {0};

        for (int i = 0; i < s.length(); i++) {
            count[s[i] - 'a']++;
            count[t[i] - 'a']--;
        }

        for (int x : count) {
            if (x != 0)
                return false;
        }

        return true;
    }
};
```

☕ Java

```java
class Solution {
    public boolean isAnagram(String s, String t) {

        if (s.length() != t.length())
            return false;

        int[] count = new int[26];

        for (int i = 0; i < s.length(); i++) {
            count[s.charAt(i) - 'a']++;
            count[t.charAt(i) - 'a']--;
        }

        for (int x : count) {
            if (x != 0)
                return false;
        }

        return true;
    }
```

```
}
```

Python

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:

        if len(s) != len(t):
            return False

        count = [0] * 26

        for i in range(len(s)):
            count[ord(s[i]) - ord('a')] += 1
            count[ord(t[i]) - ord('a')] -= 1

        return all(x == 0 for x in count)
```

JavaScript

```
var isAnagram = function(s, t) {

    if (s.length !== t.length)
        return false;

    let count = new Array(26).fill(0);

    for (let i = 0; i < s.length; i++) {
        count[s.charCodeAt(i) - 97]++;
        count[t.charCodeAt(i) - 97]--;
    }

    for (let x of count) {
        if (x !== 0)
            return false;
    }

    return true;
};
```

Interview Explanation (30 seconds)

Since the strings contain only lowercase English letters, I use a frequency array of size **26**. I increment the count for each character in the first string and decrement it for the corresponding character in the second string. If the strings are anagrams, every frequency becomes zero. This solution runs in **O(n)** time and uses **O(1)** extra space, making it the optimal approach.